

Lecture 8: Continuation Value is All You Need

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 20, 2026

University of Toronto

Neural Value Function Iteration (nVFI)

Neural Value Function Iteration (nVFI)

Broadly:

1. Parameterize $V^{\text{end}}(x^{\text{end}}, \Lambda^{\text{start}})$ using a neural network with params θ
2. Sample from Λ^{end} (somehow)
3. For each Λ^{end} compute the **outer Bellman residual** (to be defined)
4. Update θ and repeat from Step 2 until convergence

Outer Bellman Residual

Consider the Krusell-Smith model.

Let $\widehat{V}_{\theta}^{\text{end}}(x^{\text{end}}; \Lambda^{\text{end}}, Z)$ be a θ -parameterized neural network.

Informally, we want to satisfy as closely as possible,

$$\widehat{V}_{\theta}^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z) - \mathbb{E}_{Z'} \left[\widehat{V}^{\text{start}}(x^{\text{end}}; \Lambda^{\text{start}'}, Z') \mid Z \right] = 0,$$

w/ $\widehat{V}^{\text{start}}(x^{\text{end}}; \Lambda^{\text{start}'}, Z')$ given by simulating forward, then iterating backward, given $\widehat{V}_{\theta}^{\text{end}}, Z'$.

Conventions and Timing

Conventions

1. Household states do not change between household periods:

$$x_{it}^{\text{end}} = x_{i,t+1}^{\text{start}}, \quad \text{and} \quad \Lambda_{it}^{\text{end}} = \Lambda_{i,t+1}^{\text{start}}.$$

2. Aggregate shocks occur **between** household periods

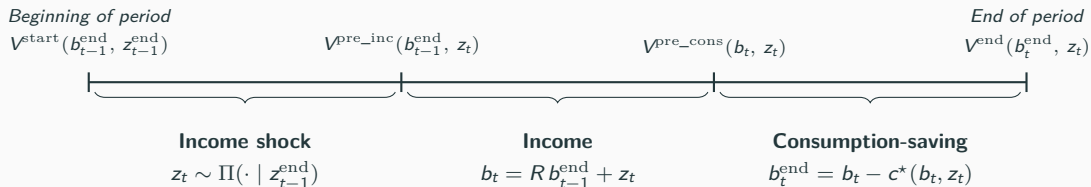
- i.e. between $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ and $V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}'}, Z')$

3. A value function **slice** or **guess** depending only on **idiosyncratic** state is denoted by tilde,

$$\tilde{V}^{\text{start}}(x^{\text{start}}), \quad \text{but} \quad V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}}, Z).$$

Within-Period Timeline: Steady State

Recall the structure of timing within a period in the steady state version:



Within-Period Timeline: Global Solution

With aggregate shocks, we rewrite (and simplify):

$$\begin{array}{ccc}
 \tilde{V}^{\text{start}}(x^{\text{start}}) & & \tilde{V}^{\text{end}}(x^{\text{end}}) \\
 \equiv V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}}, Z) & & \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)
 \end{array}$$


Λ^{start} **Household Period** Λ^{end}

In previous lectures, we learned how to map,

$$(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z) \mapsto (\tilde{V}^{\text{start}}, \Lambda^{\text{end}}).$$

Let's call this $\Phi(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z) = (\tilde{V}^{\text{start}}, \Lambda^{\text{end}}).$

Conventions: Dependence on Λ^{start}

- The model is deterministic within the household period.
- Thus, Λ^{end} is uniquely determined by Λ^{start} .
- Thus, it is **theoretically valid** to write $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$.
- **Note:** I am **not** saying anything about how to compute or represent V^{end} on the computer yet!

Conventions: Partial Application of V^{end}

- If we condition on Λ^{start} and Z , then V^{end} is just a function of x^{start} .
- That is, fixing Λ^{start} and Z , we can define,

$$\tilde{V}^{\text{end}}(x^{\text{end}}) \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z).$$

- In computer-science terms, this is a **partial application** of V^{end} at $(\Lambda^{\text{start}}, Z)$.
- Let us write this as an operator taking a **function** to a **function**

$$\mathcal{V} : (\Lambda^{\text{start}}, Z) \mapsto \tilde{V}^{\text{end}}$$

$$\mathcal{V}(\Lambda^{\text{start}}, Z)(x^{\text{end}}) \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z).$$

My view is that the **continuation value operator** $\mathcal{V}$ is a natural extension of the value function to HA models with aggregate shocks.

Continuation Value Is All You Need

Continuation Value Is All You Need

Suppose we have a model, initial aggregate state $(\Lambda^{\text{start}}, Z)$.

Suppose we also have the continuation value operator $\mathcal{V}$. (This is the hard part.)

Then we can simulate the model forward:

1. $\tilde{V}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z)$
2. $(\tilde{V}^{\text{start}}, \Lambda^{\text{end}}) = \Phi(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z)$
3. Draw $Z' \sim \Pi(\cdot \mid Z)$
4. $(\Lambda^{\text{start}'}, Z') = (\Lambda^{\text{end}}, Z')$

Recall: Typical Bellman Equation and Operator

The typical Bellman equation defines V via the relation,

$$V(b, z) = \max_{c \in [0, b]} u(c) + \beta \mathbb{E}[V(b', z') \mid z]$$

This can be thought of as a fixed point of the Bellman operator $\mathcal{T}$ which applies **lookahead**,

$$\mathcal{T}v(b, z) \equiv \max_{c \in [0, b]} u(c) + \beta \mathbb{E}[v(b', z') \mid z]$$

So that the Bellman equation is,

$$V = \mathcal{T}V.$$

I define a similar **outer Bellman equation**.

The Outer Bellman Equation

The true $\mathcal{V}$ also satisfies a recursive equation, the **outer Bellman equation**:

$$\mathcal{V}^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z) = \mathbb{E}_Z \left[\tilde{\mathcal{V}}^{\text{start}'}(x^{\text{end}}) \mid Z \right],$$

where

$$\tilde{\mathcal{V}}^{\text{start}'} = \Phi_V(\Lambda^{\text{end}}, \tilde{\mathcal{V}}^{\text{end}'}, Z'),$$

$$\tilde{\mathcal{V}}^{\text{end}'} = \mathcal{V}(\Lambda^{\text{end}}, Z'),$$

$$Z' \sim \Pi_Z(\cdot \mid Z),$$

$$\Lambda^{\text{end}} = \Phi_\Lambda(\Lambda^{\text{start}}, \tilde{\mathcal{V}}^{\text{end}}, Z),$$

$$\tilde{\mathcal{V}}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z).$$

The Outer Bellman Operator

For any $\mathcal{V}_\theta$, we can define the **outer Bellman operator** as:

$$\mathcal{L}\mathcal{V}_\theta(\Lambda^{\text{start}}, Z)(x^{\text{end}}) \equiv \mathbb{E}_Z \left[\tilde{V}^{\text{start}'}(x^{\text{end}}) \mid Z \right],$$

where

$$\tilde{V}^{\text{start}'} = \Phi_V(\Lambda^{\text{end}}, \tilde{V}^{\text{end}'}, Z'),$$

$$\tilde{V}^{\text{end}'} = \mathcal{V}_\theta(\Lambda^{\text{end}}, Z'),$$

$$Z' \sim \Pi_Z(\cdot \mid Z),$$

$$\Lambda^{\text{end}} = \Phi_\Lambda(\Lambda^{\text{start}}, \tilde{V}^{\text{end}}, Z),$$

$$\tilde{V}^{\text{end}} = \mathcal{V}_\theta(\Lambda^{\text{start}}, Z).$$

The Outer Bellman Residual

The true **continuation value operator** $\mathcal{V}$ is defined by,

$$\mathcal{L}\mathcal{V} = \mathcal{V}.$$

For any $\mathcal{V}_\theta$, we can define the **outer Bellman residual** as,

$$\mathcal{L}\mathcal{V}_\theta - \mathcal{V}_\theta,$$

and we can frame the problem of approximating $\mathcal{V}$ as,

$$\min_{\theta} \|\mathcal{L}\mathcal{V}_\theta - \mathcal{V}_\theta\|^2.$$

Implementing CVIAYN

What we inherit from L7

The L7 K-S notebook already gives us everything except the network and the training loop:

- `KSPParams` (§3 of L7).
- `ks_prices(K, Z, p)` (§4 of L7).
- The 3-stage *simulation* chain `ks_household_sim = z_shock ○ income ○ savings` (§8 of L7).
- `aiyagari_steady_state_at_Z` (§5 of L7) — gives us V_{ss} for pretraining and Λ_{ss} as the sampler's initial condition.

What we drop: the `K_evolve + Z_shock` chain tail (the agent's ALM belief) *and* the K -grid in V . The network replaces both.

Φ in code: just the simulation chain

The 3-stage chain *is* our Φ . Given any $\tilde{V}^{\text{end}}$ over the idiosyncratic (b, z) grid, it

- runs backward to get $\tilde{V}^{\text{start}}$ (savings stage solves the household's choice; income / z_shock stages compose);
- runs forward to push μ^{start} to μ^{end} .

The only thing it cannot do alone is supply $\tilde{V}^{\text{end}}$. That's what $\mathcal{V}_\theta$ provides.

Architecture: how does Λ get in?

We can't dump an $N_b \times N_z$ field into a dense layer and learn anything. The standard fix (Han–Yang–E 2025): summarise Λ as a small *learned* vector before the value head sees it.

Four sub-nets:

1. $\text{pop_agg_net} : (b, z) \rightarrow \mathbb{R}^k$ — embed each cell.
2. $\text{GM}(\Lambda) = \sum_{(b,z)} \Lambda(b, z) \cdot \text{pop_agg_net}(b, z) \in \mathbb{R}^k$ — population-weighted mean of embeddings. One matmul.
3. $V_{\text{pre}} : (b, z) \rightarrow \mathbb{R}^m$ — cell-level features for the value head.
4. $V_{\text{post}} : \mathbb{R}^{m+N_z} \rightarrow \mathbb{R}$ — combines V_{pre} 's features with the (projected) GM and the Z one-hot.

Combination is **additive**: $V_{\text{pre}}(\text{cell}) + \text{gm_post}(\text{GM})$, then concatenate Z .

The network in code

```
struct VNet{A, P, G, V}
    pop_agg_net::A    # 2 -> gm_dim        per-cell embedding
    V_pre::P          # 2 -> mid           per-cell features
    gm_post::G         # gm_dim -> mid      project GM to mid
    V_post::V          # mid + N_Z -> 1     combine + Z
end

function (net::VNet)Λ(, Z_idx, p)
    pop_emb = net.pop_agg_net(HH_FEAT)      # (gm_dim, N_cells)
    gm       = pop_emb * Float32.(vecΛ())    # (gm_dim,)
    h        = net.V_pre(HH_FEAT) .+ net.gm_post(gm) # (mid, N_cells)
    pred     = net.V_post(vcat(h, Z_ROWS[Z_idx])) # (1, N_cells)
    return reshape(pred, p.N_w, length(p.z_grid))
end
```

The bottleneck $k = \text{gm_dim}$ replaces the $N_b N_z$ -dim input with a learned k -dim summary.

One inner solve under V_θ

```
function inner_solve(hh, vnet::VNet,  $\Lambda$ ::AbstractMatrix, Z_idx::Int, p)
    V_end = Float64.(vnet $\Lambda$ (, Z_idx, p)) # N_b x N_z
    env    = make_env(hh; ks_prices(integrate_K(hh,  $\Lambda$ ),
                                     p.Z_vals[Z_idx], p)...)
    return backward!(hh, V_end, env)
end
```

- Network predicts $\widehat{V}_\theta^{\text{end}}$ at (Λ, Z) .
- Build env at prices $(r, w) = \text{ks_prices}(K(\Lambda), Z, p)$. Prices depend on Λ only through $K = \int b d\Lambda$ — that's the production technology, not the network's architecture.
- One backward sweep through the 3-stage chain $\Rightarrow \widetilde{V}^{\text{start}}$.

The lookahead operator $\mathcal{L}V_\theta$

```
function lookahead(hh, vnet,  $\Lambda$ _start, Z_idx, p)
    # Forward one period under  $V_\theta$  to get  $\Lambda_{end}$ .
    V_end_now = Float64.(vnet( $\Lambda$ _start, Z_idx, p))
    env_now    = make_env(hh; ks_prices(integrate_K(hh,  $\Lambda$ _start),
                                                    p.Z_vals[Z_idx], p)...)

    backward!(hh, V_end_now, env_now)  $\Lambda$ 
    _end = forward!(hh,  $\Lambda$ _start)

    # Backward at each next-period Z; take expectation over Z.
    V_end_pred = zeros(p.N_w, length(p.z_grid))
    for Z_next in 1:N_Z
        V_start_next = inner_solve(hh, vnet,  $\Lambda$ _end, Z_next, p)
        V_end_pred .+= p.P_Z[Z_idx, Z_next] .* V_start_next
    end
    return V_end_pred
end
```

What just happened

We computed $\mathcal{L}V_\theta$ at one sample Λ_i^{start} with aggregate index Z_i :

1. Network predicts $\hat{V}_\theta^{\text{end}}$ at $(\Lambda_i^{\text{start}}, Z_i)$.
2. Φ pushes Λ_i^{start} forward $\rightarrow \Lambda_i^{\text{end}}$.
3. For each Z_{next} , network predicts $\hat{V}_\theta^{\text{end}}$ at $(\Lambda_i^{\text{end}}, Z_{\text{next}})$. Backward through Φ gives $V_{\text{next}}^{\text{start}}$.
4. Take expectation over Z_{next} using $P_Z[Z_i, \cdot]$.

Result: a (b, z) -shaped array, $\hat{V}_i^{\text{end}}$. This is our training label.

The training loss

```
function bellman_residual_loss(vnet, batch, p, hh)
    L = 0.0
    for  $\Lambda$ (_start, Z_idx) in batch
        target = lookahead(hh, vnet,  $\Lambda$ _start, Z_idx, p)
        pred    = vnet( $\Lambda$ (_start, Z_idx, p)
        L += mean((pred .- Float32.(target)).^2) / var_scale(target)
    end
    return L / length(batch)
end
```

Stop-gradient on the label. The lookahead is computed once and treated as a fixed target. We do *not* differentiate through Φ (the within-period solver). Only V_θ 's direct dependence on θ enters the gradient.

Sampling Λ^{start}

```
function sample_ergodic(hh, vnet, p; T=200, burn=100, every=2, rng=...)Λ
, Z_idx = copyΛ(_ss), 1
batch = Tuple{Matrix{Float64}, Int}[]
for t in 1:T
    V_end = Float64.(vnetΛ(, Z_idx, p))
    env    = make_env(hh; ks_prices(integrate_K(hh, Λ),
                                                p.Z_vals[Z_idx], p)...)
    backward!(hh, V_end, env); Λ = forward!(hh, Λ)
    Z_idx = sample_Z(Z_idx, p.P_Z, rng)
    t > burn && (t - burn) % every == 0 && push!(batch, (copyΛ(), Z_idx))
end
return batch
end
```

MCMC-style: simulate under the *current* V_θ , burn in, subsample.

The training loop

```
function train!(vnet, p, hh; epochs=50, lr=1e-3, batch_kwargs=(; T=200, burn=100))  
    opt = Flux.setup(Adam(lr), vnet)  
    hist = Float64[]  
    for epoch in 1:epochs  
        batch = sample_ergodic(p, hh, vnet; batch_kwargs...)  
        loss, grads = Flux.withgradient(vnet) do net  
            bellman_residual_loss(net, batch, p, hh)  
        end  
        Flux.update!(opt, vnet, grads[1])  
        push!(hist, loss)  
    end  
    return hist  
end
```

Initialization matters

A randomly-initialized V_θ usually produces a within-period solve that fails (numerically unbounded consumption, etc.).

Pretrain. Match the deterministic value function V_{ss} across a cloud of Λ 's (perturbed Λ_{ss}) and all Z 's. Not because V_{ss} is right at any $\Lambda \neq \Lambda_{ss}$ — it isn't — but because it's a sensible *constant baseline* that gets V_θ into the right scale and shape before the residual training loop starts moving things around.

Cheap, no L7 dependency.

Running it

```
p    = KSParams()
hh   = ks_household_2d(p)
vnet = VNet()
pretrain_to_ss!(vnet, V_ss, p)

hist = train!(vnet, hh, p; epochs=50)
plot(hist; yaxis=:log, lw=2,
      xlabel="epoch", ylabel="outer Bellman residual")
```

Honest disclaimer. 50 epochs on a CPU is a *demonstration*, not a converged solve. Real runs take many thousands of epochs on a GPU. See §7.

Reading the Answer

The training curve

- Outer Bellman residual on log scale, vs. training epoch.
- Expect a noisy but downward-trending curve.
- In a real run (Peking 2025 deck, §5): NN reaches $\approx 10^{-4.4}$ relative variance; K-S baseline plateaus at $\approx 10^{-3.2}$.

The gap is the headline: at convergence, V_θ explains the residual variance *better* than K-S does, on the same problem, with no parametric ansatz.

What "outer Bellman residual" means as an error

A small residual means V_θ is approximately a fixed point of $\mathcal{L}$.

By a contraction argument (standard, when $\mathcal{L}$ is a β -contraction), this implies $\|V_\theta - V\| \leq \frac{1}{1-\beta} \|V_\theta - \mathcal{L}V_\theta\|$. Small residual $\Rightarrow$ small distance to truth.

Caveat. The contraction property of $\mathcal{L}$ for this setup is a research-grade question; the paper appendix sketches conditions. For our K-S baseline, the empirical contraction holds.

Comparison with K-S

Three ways to compare V_θ to L7's K-S baseline:

1. **Value-function MSE** on a common (b, z, Λ_t, Z_t) simulation path. K-S evaluates at K_t ; CVIAYN evaluates at Λ_t .
2. **Policy MSE**: extract b' from each, compare.
3. **Den Haan error**: simulate aggregate K_t paths under each. CVIAYN matches K-S up to noise here because the K-S *ansatz* happens to be near-true on the baseline calibration (see L7 §6).

The interesting deltas appear in (1) and (2) at tail states (rare Λ , rare Z): K-S extrapolates the linear ALM; CVIAYN extrapolates the network.

Where the K–S equivalence breaks

K–S looks indistinguishable from CVIAYN *on the baseline calibration*. That's because the conditional law of motion of K happens to be approximately log-linear — and the population marginal we care about for prices, K , happens to be a single scalar.

Settings where CVIAYN pulls away:

- Multi-asset (illiquid wealth): non-log-linear dynamics.
- Discrete choice (sector, occupation): occupancy shares aren't moments of K .
- Spatial: distribution *across* locations is not summarized by aggregates.

In each case, K–S would need a hand-engineered ansatz tracking extra moments; CVIAYN just grows the GM input.

Generalizations

Where this matters

Models for which the scalar K ansatz *cannot* work:

- Search-and-matching (vacancy distribution).
- OLG (age structure of the cross-section).
- Spatial (where the population is, not just how much).
- Multi-asset (liquid vs. illiquid composition).
- Discrete choice (occupational mix).

CVIAYN with the GM input scales to all of these without changing the algorithm. The recipe grows along well-understood axes: more cell features, per-location GMs, per-age GMs.

How the GM architecture extends

Per-location GM (spatial / urban models). Build one GM_ℓ per location ℓ from the location's own Λ -slice. An aggregate-of-aggregates net summarises across locations.

Age stratification (OLG). Run a separate `pop_agg_net` per cohort, or share weights and feed age as an extra cell feature.

Multi-asset. The state space gains a dimension; `pop_agg_net` takes (b_{liq}, b_{illi}, z) as input. Nothing else changes.

Non-population state. Spin up an `env_net` that embeds exogenous aggregate state into a parallel GM; add to the cell-level features the same way.

The optimized reference implementation

The notebook's training loop is honest but pedagogical. A production-grade implementation lives in this repository at

- `reference_materials/
example_usage/stochastic_transition/code/
stochastic_transition/`

It adds, on top of today's structure: per-location GM + aggregate-of-aggregates head; a parallel `env_net` for non-population state; cosine-annealed LR; low-memory gradient accumulation; caching of within-period solves along the simulation path; per-location backprop with shared GMs.

The *algorithm* is unchanged; everything is plumbing.

One-step lookahead is a knob

The lookahead in §5 was one period: solve Φ at Λ^{start} , draw Z_{next} , solve Φ at $\Lambda^{\text{start}'}$, take expectation.

n -step lookahead. Do n rounds of (Φ, Ω) inside the operator. Equivalent to applying $\mathcal{L}^n$.

- Pro: lower bias on the residual target (deeper lookahead $\rightarrow$ truer label).
- Con: cost grows linearly in n ; variance grows in the number of Z paths.

The notebook uses $n = 1$. Higher n is a one-line change.

Three short appendices on:

- why the timing convention matters,
- why we stop-gradient on the lookahead label,
- related literature.

Why the timing convention matters

The split into V^{start} vs. V^{end} is what makes $\mathcal{V}$ a clean object.

Without the split: $V(x^{\text{start}}, \Lambda^{\text{start}})$ depends on Λ^{start} , but also implicitly forecasts everything that happens *within* t . Hard to disentangle "what the network learns" from "what Φ computes."

With the split: V^{end} lives at the moment *just before* the next aggregate shock. Everything inside the period is handled by Φ ; everything across periods is handled by $\mathcal{V}$.

Why stop-gradient on the label

The label $\widehat{V}^{\text{end}}_i$ is itself computed using V_θ (via the lookahead). One could backprop through that too.

Why we don't:

- Each lookahead involves several runs of Φ , which involves gridsearch, monotone-search tricks, and other non-differentiable pieces.
- The "loss-as-fixed-point-residual" formulation is the standard ML treatment of value iteration; SARSA, Q-learning, etc. all stop gradients on bootstrapped targets.
- Empirically, semi-gradient stabilizes training.

Related literature

Projection / local perturbation: Bhandari et al. (2023), Bilal (2023), Auclert et al. (2021), Winberry (2018).

Deep policy approximation: Han et al. (2024), Azinovic et al. (2022), Maliar et al. (2021), Fernández-Villaverde et al. (2023), Gu et al. (2024).

Non-neural value approximation: Krusell & Smith (1998), Hull (2015).

Generalized moments architecture: Han, Yang & E (2025).

CVIAYN extends the third with deep learning, drawing on the GM construction from the fourth.